NTFS File Server Lab

A step by step Azure and Terraform lab, written so a 12 year old can follow along

Active Directory | NTFS Permissions | SMB File Services | Group Policy | Terraform | PowerShell

This is the merged, corrected final version. It combines the full walkthrough with the fixes found during review, so every command in this document is the safe, working version, not the original with its bugs left in.

REVIEWED (this pass): every Terraform file in this document had a formatting bug where several attributes were crammed onto one line separated by commas, for example { type = string, default = "Central US" }. That comma style is not valid Terraform syntax and terraform init or terraform plan would have failed on it. Every block below has been rewritten one attribute per line, which is the only format Terraform actually accepts. Two small PowerShell issues were also fixed: a stray backtick before a colon in the icacls grant line in script 02, and a risky New-Object PSCredential(...) constructor call in script 04 that has been replaced with the standard -ArgumentList form.

What Are We Even Doing Here?

Picture an office building with a Finance department, an HR department, a Sales department, and an IT department. Each one has its own filing cabinet, and the rule is simple: Finance staff can open the Finance cabinet, HR staff can open the HR cabinet, and nobody outside IT can open every cabinet in the building.

In this lab, we build exactly that, except the filing cabinets are shared folders on a Windows file server, and the rules about who can open what are enforced by two computers working together: one that keeps the list of every employee and which department they belong to, and one that holds the actual folders and checks that list before letting anyone in.

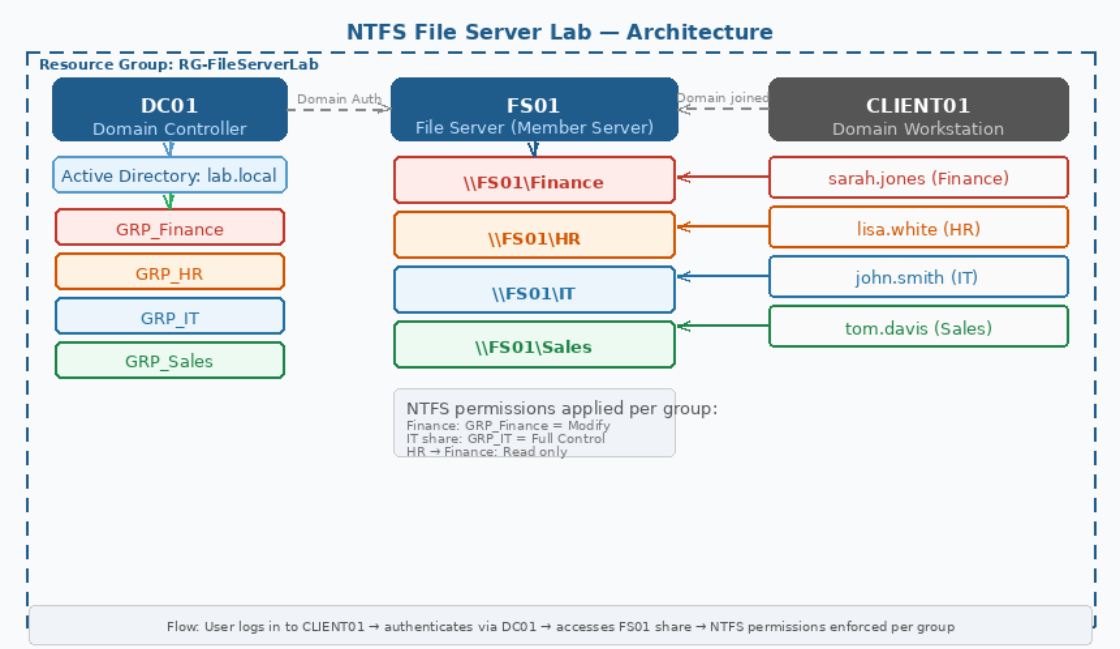
We build all of it with Terraform, a tool that reads text files describing what we want and builds the real thing in Azure for us, the same LEGO instruction booklet idea as the domain controller lab: we write down exactly what to build, then hand the instructions to a robot that builds it.

What this actually means:

- A domain controller is the computer that keeps the master list of every user and every rule. Everything else checks in with it before deciding who's allowed to do what.
- NTFS is the actual lock on the door, the system Windows uses to decide who can open, change, or delete a specific file or folder.
- SMB is the hallway that lets computers share folders over a network, the protocol behind every mapped network drive you've ever used.
- Group Policy is a rule that gets pushed out automatically to a whole group of computers at once, like a school-wide announcement instead of telling each classroom individually.

Architecture: What Gets Built

DC01 runs Active Directory, DNS, and Group Policy, the front desk and rulebook computer. FS01 hosts four SMB shares with NTFS permissions enforced per security group, the filing cabinet room. CLIENT01 is the Windows 11 workstation where test users log in to prove the whole permission chain actually works. All three computers sit on the same private network, protected by a security rule that only allows remote connections from your own address.



Why Each Piece Exists

Piece	What it does and why it's needed
Resource Group	One labeled container for everything in this lab, so deleting it cleans up all of it at once.
VNet and Subnet	A private, fenced-off area where all three computers live and can reach each other without going through the public internet.
Security rule (NSG)	A bouncer that only lets remote connections in from your own address, not the whole internet.
Static address on DC01	DC01 always has the same private address, so the other two computers can always find it to ask their questions.
Public addresses	Let you personally connect in from your own computer to check on things.
Key Vault	A digital safe that stores the admin password so it never sits in a plain text file or shows up in your terminal history.

Before You Start: Prerequisites

Make sure these are ready on your own computer before starting. Open a terminal (PowerShell on Windows, Terminal on Mac) and check each one:

```
terraform -version   # Must be 1.5.0 or newer az version            # Any recent Azure
CLI version powershell -version   # 5.1 or newer, already built into Windows
```

Then confirm you're pointed at the right Azure subscription, since that's what gets billed:

```
az account show # If it's the wrong one: az account set --subscription "name or ID"
```

Step 1: One Time Remote State Setup

Before writing any Terraform files, create a storage box for Terraform's own notes about what it's already built. This step is moved ahead of writing the files, unlike the original draft, so you never reference a storage box that doesn't exist yet.

FIXED: moved this step to be first. The original draft had you write backend.tf, which mentions this storage account by name, before the storage account existed yet. That created a confusing detour where you had to write a file, skip ahead, then come back and edit it.

```
az group create --name RG-TerraformState --location "Central US"  az storage account create \   --name tfstatentfslab \   --resource-group RG-TerraformState \   --sku Standard_LRS \   --encryption-services blob  az storage container create --name tfstate --account-name tfstatentfslab  # Verify, must show: tfstate az storage container list --account-name tfstatentfslab --query "[].name" -o tsv
```

What this actually means:

- Terraform keeps track of every resource it creates in a file called state. If that file only lives on your own laptop and you lose it, Terraform loses track of everything it built, making cleanup very difficult.
- Storing that state file in Azure Blob Storage instead means it's encrypted, backed up, and reachable from any computer, not just this one.

Step 2: Create Your Project Folder

Create a dedicated folder for this lab. All Terraform files go in the root, and PowerShell scripts go in a scripts subfolder, since the orchestration script refers to that scripts folder by a relative path.

```
New-Item -ItemType Directory -Path "$HOME\ntfs-lab-terraform" cd "$HOME\ntfs-lab-terraform" New-Item -ItemType Directory -Path scripts
```

By the end of this lab your folder will look like this:

```
ntfs-lab-terraform/   backend.tf              remote state, points at the storage box from Step 1   versions.tf             which provider versions Terraform downloads   variables.tf             every input variable, including admin_password   main.tf   VMs, VNet, NSG, NICs, public IPs   keyvault.tf              Key Vault, secret, and RBAC assignment   outputs.tf               IPs and Key Vault name printed after apply   terraform.tfvars.example  a safe template, fine to commit to git   terraform.tfvars   your real values, never commit this   .gitignore               stops the sensitive files above from being committed   configure-lab.ps1         the only script you run by hand   scripts/     00-promote-dc.ps1     01-create-ad-users-groups.ps1     02-configure-shares-and-permissions.ps1     03-configure-rdp-gpo.ps1     04-domain-join.ps1     05-verify-ad.ps1     05-verify-shares.ps1     06-add-rdp-users.ps1
```

Step 3: Write the Terraform Files

Open each file below in a plain text editor (not Microsoft Word) and paste in its matching content.

backend.tf

This points Terraform at the storage box you created in Step 1.

```
terraform {   backend "azurerm" {     resource_group_name  = "RG-TerraformState"   storage_account_name = "tfstatentfslab"     container_name        = "tfstate"     key = "ntfs-lab.terraform.tfstate"   } }
```

versions.tf

Provider	Why this lab needs it
azurerm	The Azure provider, creates every Azure resource: VMs, VNet, Key Vault, NSG, and so on.
random	Generates a random 8 character suffix for the Key Vault name so it's unique across all of Azure.
time	Provides pauses after network resources are created, to avoid Azure's occasional replication delays.

```
terraform {   required_version = ">= 1.5.0"   required_providers {     azurerm =
{ source = "hashicorp/azurerm", version = "~> 3.0" }     random  = { source =
"hashicorp/random",  version = "~> 3.6" }     time    = { source = "hashicorp/time",
version = "~> 0.11" }   } } provider "azurerm" { features {} }
```

variables.tf

admin_password has no default and is marked sensitive, so Terraform never prints it, and it must be provided as an environment variable, never written into a file.

```
variable "location" {   type        = string   default     = "Central US"
description = "Azure region." } variable "resource_group_name" {   type    = string
default = "RG-FileServerLab" } variable "vnet_name" {   type    = string   default =
"VNET-FileServerLab" } variable "subnet_name" {   type    = string   default =
"Subnet-Servers" } variable "vnet_cidr" {   type    = string   default = "10.0.0.0/16"
} variable "subnet_cidr" {   type    = string   default = "10.0.1.0/24" } variable
"nsg_name" {   type    = string   default = "NSG-RDP" } variable "rdp_source" {   type
= string   description = "Your public IP in CIDR format, e.g. 1.2.3.4/32. Find it at
whatismyip.com. Never leave this as a wildcard." } variable "admin_username" {   type
= string   default = "azureadmin" } variable "admin_password" {   type        = string
sensitive   = true   description = "Set as TF_VAR_admin_password environment variable,
never in a file." } variable "server_vm_size" {   type    = string   default =
"Standard_B2as_v2" } variable "client_vm_size" {   type    = string   default =
"Standard_B2as_v2" }
```

FIXED: rdp_source has no default value here on purpose, unlike a version of this file that could default it to a wildcard "*". Terraform will refuse to build anything until you provide your own IP address in terraform.tfvars, so the front door can never accidentally get left open to the whole internet.

main.tf

Creates all networking and VMs. Every time_sleep pause exists because Azure sometimes has brief replication delays right after network resources are created, and resources built immediately after can fail with a transient error without that pause.

```
resource "azurerm_resource_group" "rg" {   name     = var.resource_group_name
location = var.location }  resource "azurerm_virtual_network" "vnet" {   name
= var.vnet_name   location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   address_space       = [var.vnet_cidr] }  resource
"time_sleep" "wait_after_vnet" {   create_duration = "45s"   depends_on      =
[azurerm_virtual_network.vnet] }  resource "azurerm_subnet" "subnet" {   name
= var.subnet_name   resource_group_name  = azurerm_resource_group.rg.name
virtual_network_name = azurerm_virtual_network.vnet.name   address_prefixes     =
[var.subnet_cidr]   depends_on           = [time_sleep.wait_after_vnet] }  # NSG, only
your IP can reach port 3389. Everything else is denied by default. resource
"azurerm_network_security_group" "nsg" {   name                = var.nsg_name
location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   security_rule {     name                       =
"Allow-RDP-3389"     priority                   = 100     direction                =
"Inbound"     access                     = "Allow"     protocol                   =
"Tcp"     source_address_prefix      = var.rdp_source     source_port_range         =
```

```
"*"     destination_port_range     = "3389"     destination_address_prefix = "*"   }
depends_on = [time_sleep.wait_after_vnet] }  resource "time_sleep" "wait_after_nsg" {
create_duration = "45s"   depends_on      = [azurerm_network_security_group.nsg] }
resource "azurerm_public_ip" "dc01" {   name                = "dc01-pip"   location
= var.location   resource_group_name = azurerm_resource_group.rg.name
allocation_method   = "Static"   sku                 = "Standard"   depends_on
= [time_sleep.wait_after_nsg] } resource "azurerm_public_ip" "fs01" {   name
= "fs01-pip"   location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   allocation_method   = "Static"   sku
= "Standard"   depends_on          = [time_sleep.wait_after_nsg] } resource
"azurerm_public_ip" "client01" {   name                = "client01-pip"   location
= var.location   resource_group_name = azurerm_resource_group.rg.name
allocation_method   = "Static"   sku                 = "Standard"   depends_on
= [time_sleep.wait_after_nsg] }  # DC01 NIC, static IP 10.0.1.4 so FS01/CLIENT01 DNS
never breaks after a restart resource "azurerm_network_interface" "dc01" {   name
= "dc01-nic"   location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   ip_configuration {     name
= "internal"     subnet_id                     = azurerm_subnet.subnet.id
private_ip_address_allocation = "Static"     private_ip_address            =
"10.0.1.4"     public_ip_address_id          = azurerm_public_ip.dc01.id   }
depends_on = [time_sleep.wait_after_nsg] } resource "azurerm_network_interface" "fs01"
{   name                = "fs01-nic"   location            = var.location
resource_group_name = azurerm_resource_group.rg.name   ip_configuration {     name
= "internal"     subnet_id                     = azurerm_subnet.subnet.id
private_ip_address_allocation = "Dynamic"     public_ip_address_id          =
azurerm_public_ip.fs01.id   }   depends_on = [time_sleep.wait_after_nsg] } resource
"azurerm_network_interface" "client01" {   name                = "client01-nic"
location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   ip_configuration {     name
= "internal"     subnet_id                     = azurerm_subnet.subnet.id
private_ip_address_allocation = "Dynamic"     public_ip_address_id          =
azurerm_public_ip.client01.id   }   depends_on = [time_sleep.wait_after_nsg] }  #
Attach the NSG to each NIC, without this the NSG exists but protects nothing resource
"azurerm_network_interface_security_group_association" "dc01" {   network_interface_id
= azurerm_network_interface.dc01.id   network_security_group_id =
azurerm_network_security_group.nsg.id   depends_on = [time_sleep.wait_after_nsg] }
resource "azurerm_network_interface_security_group_association" "fs01"
{   network_interface_id = azurerm_network_interface.fs01.id
network_security_group_id = azurerm_network_security_group.nsg.id   depends_on =
[time_sleep.wait_after_nsg] } resource
"azurerm_network_interface_security_group_association" "client01"
{   network_interface_id = azurerm_network_interface.client01.id
network_security_group_id = azurerm_network_security_group.nsg.id   depends_on =
[time_sleep.wait_after_nsg] }  # DC01, Windows Server 2022 Azure Edition (comes with
the VM agent pre-installed) resource "azurerm_windows_virtual_machine" "dc01" {   name
= "DC01"   location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   size                = var.server_vm_size
admin_username      = var.admin_username   admin_password      = var.admin_password
network_interface_ids = [azurerm_network_interface.dc01.id]   os_disk {     caching
= "ReadWrite"     storage_account_type = "Standard_LRS"   }   source_image_reference {
publisher = "MicrosoftWindowsServer"     offer     = "WindowsServer"     sku       =
"2022-datacenter-azure-edition"     version   = "latest"   }   depends_on =
[time_sleep.wait_after_nsg] }  # FS01, Windows Server 2022 resource
"azurerm_windows_virtual_machine" "fs01" {   name                = "FS01"   location
= var.location   resource_group_name = azurerm_resource_group.rg.name   size
= var.server_vm_size   admin_username      = var.admin_username   admin_password
= var.admin_password   network_interface_ids = [azurerm_network_interface.fs01.id]
os_disk {     caching              = "ReadWrite"     storage_account_type =
"Standard_LRS"   }   source_image_reference {     publisher = "MicrosoftWindowsServer"
offer     = "WindowsServer"     sku       = "2022-datacenter-azure-edition"
version   = "latest"   }   depends_on = [time_sleep.wait_after_nsg] }  # CLIENT01,
```

```
Windows 11 Pro (RDP is off by default, the extension below turns it on) resource
"azurerm_windows_virtual_machine" "client01" {   name                = "CLIENT01"
location            = var.location   resource_group_name =
azurerm_resource_group.rg.name   size                = var.client_vm_size
admin_username      = var.admin_username   admin_password      = var.admin_password
network_interface_ids = [azurerm_network_interface.client01.id]   os_disk
{     caching              = "ReadWrite"     storage_account_type = "Standard_LRS"   }
source_image_reference {     publisher = "MicrosoftWindowsDesktop"     offer     =
"windows-11"     sku       = "win11-23h2-pro"     version   = "latest"   }
depends_on = [time_sleep.wait_after_nsg] }  # Enable RDP on CLIENT01, Windows 11 ships
with it off. Without this, RDP attempts # to CLIENT01 time out with no error message
at all. resource "azurerm_virtual_machine_extension" "client01_enable_rdp" {   name
= "enable-rdp"   virtual_machine_id   = azurerm_windows_virtual_machine.client01.id
publisher            = "Microsoft.Compute"   type                 =
"CustomScriptExtension"   type_handler_version = "1.10"   settings =
jsonencode({     commandToExecute = "powershell -Command \"Set-ItemProperty -Path
'HKLM:\\System\\CurrentControlSet\\Control\\Terminal Server' -Name
'fDenyTSConnections' -Value 0; Enable-NetFirewallRule -DisplayGroup 'Remote
Desktop'\""   })   depends_on = [azurerm_windows_virtual_machine.client01] }
```

Why terraform plan should show about 22 to 24 resources here, not 12 to 15:

• Counting every resource above (resource group, VNet, 2 pauses, subnet, NSG, 3 public IPs, 3 NICs, 3 NSG attachments, 3 VMs, 1 extension) plus the Key Vault pieces in the next file gives roughly 22 to 24, not the smaller number an earlier version of this document mentioned. If your plan shows a number in that range, nothing is wrong.

keyvault.tf

Creates the Key Vault and stores the admin password as a secret. The orchestration script retrieves this at runtime, so you never type the password again after terraform apply finishes.

```
data "azurerm_client_config" "current" {}  resource "random_id" "kv_suffix"
{ byte_length = 4 }  resource "azurerm_key_vault" "lab_kv" {   name
= "kv-fslab-${random_id.kv_suffix.hex}"   location                  =
azurerm_resource_group.rg.location   resource_group_name        =
azurerm_resource_group.rg.name   tenant_id                  =
data.azurerm_client_config.current.tenant_id   sku_name                  = "standard"
enable_rbac_authorization  = true   # required, without this every secret read/write
returns 403   soft_delete_retention_days = 7   purge_protection_enabled   = false
tags = { Environment = "Lab", ManagedBy = "Terraform" } }  # Grant whoever ran az
login permission to write secrets resource "azurerm_role_assignment"
"kv_deployer_access" {   scope                = azurerm_key_vault.lab_kv.id
role_definition_name = "Key Vault Secrets Officer"   principal_id          =
data.azurerm_client_config.current.object_id }  resource "azurerm_key_vault_secret"
"admin_password" {   name         = "vm-admin-password"   value        =
var.admin_password   key_vault_id = azurerm_key_vault.lab_kv.id   depends_on   =
[azurerm_role_assignment.kv_deployer_access]   tags         = { ManagedBy =
"Terraform" } }
```

FIXED: if you ever manually check IAM on this vault and go looking for a role called "Key Vault Secrets User," you won't find it, and that's fine. This lab grants "Key Vault Secrets Officer" instead, which already includes everything Secrets User can do, plus the ability to write the password in the first place.

outputs.tf

```
output "dc01_public_ip" { value = azurerm_public_ip.dc01.ip_address } output
"dc01_private_ip" { value = azurerm_network_interface.dc01.private_ip_address } output
"fs01_public_ip" { value = azurerm_public_ip.fs01.ip_address } output
"client01_public_ip" { value = azurerm_public_ip.client01.ip_address } output
"key_vault_name" { value = azurerm_key_vault.lab_kv.name }
```

terraform.tfvars.example

A safe template with no real secrets in it, fine to commit to git as-is.

```
location        = "Central US" rdp_source      = "YOUR_PUBLIC_IP/32"   # find yours at whatismyip.com, format 1.2.3.4/32 server_vm_size  = "Standard_B2as_v2" client_vm_size = "Standard_B2as_v2" # admin_password is NOT set here, use an environment variable instead: # PowerShell: $env:TF_VAR_admin_password = "YourStrongPassword!" # Min 12 characters, upper + lower + number + symbol.
```

.gitignore

```
terraform.tfvars *.tfvars !terraform.tfvars.example terraform.tfstate terraform.tfstate.backup *.tfstate .terraform/ .terraform.lock.hcl *.tfplan .DS_Store Thumbs.db
```

Step 4: Configure Your Variables

```
Copy-Item terraform.tfvars.example terraform.tfvars # Open terraform.tfvars and set rdp_source to your own IP from whatismyip.com $env:TF_VAR_admin_password = "YourStrongPassword!" echo $env:TF_VAR_admin_password   # if nothing prints, set it again before apply
```

Never leave rdp_source unset or as a wildcard:

- This is the single setting that decides whether the front door is open to just you or to the entire internet. Always double check it before running apply.

Step 5: Deploy the Infrastructure

```
az login terraform init    # downloads providers, connects to remote state terraform plan    # review, expect about 22 to 24 resources including 3 VMs terraform apply   # type yes, takes 10 to 15 minutes  # After apply, copy the Key Vault name for Step 8 terraform output key_vault_name
```

Output to write down	Used for
key_vault_name	Step 8, the orchestration script needs this
client01_public_ip	Step 9, for RDP testing

Step 6: Create the Scripts

Create each file inside your scripts folder. These get pushed to the right VM automatically by the orchestration script in Step 7, you never run them by hand.

scripts/00-promote-dc.ps1

Installs AD DS and promotes DC01 to a Domain Controller for lab.local. DC01 reboots automatically after this, and the orchestration script waits for it to come back online before moving on.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force Write-Host
"Installing AD DS..." -ForegroundColor Yellow Install-WindowsFeature -Name AD-Domain-
Services -IncludeManagementTools -Verbose:$false $safeModePassword = ConvertTo-
SecureString "SAFE_MODE_PASSWORD" -AsPlainText -Force Import-Module ADDSDeployment
Install-ADDSForest `   -DomainName "lab.local" `   -DomainNetbiosName "LAB" `   -
ForestMode "WinThreshold" `   -DomainMode "WinThreshold" `   -InstallDns:$true `   -
SafeModeAdministratorPassword $safeModePassword `   -Force:$true `   -
NoRebootOnCompletion:$false # DC01 reboots here. The orchestration script waits for it
to come back online.
```

scripts/01-create-ad-users-groups.ps1

Creates three OUs, four security groups, and five test users. Group membership drives NTFS permissions, add a user to GRP_Finance and they automatically inherit Finance share access with no other changes needed.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $domain =
"lab.local"; $domainDN = "DC=lab,DC=local" $password = ConvertTo-SecureString
"P@ssw0rd123!" -AsPlainText -Force  foreach ($ou in @("Lab Users","Lab Computers","Lab
Groups")) {   New-ADOrganizationalUnit -Name $ou -Path $domainDN -
ProtectedFromAccidentalDeletion $false   Write-Host "Created OU: $ou" -ForegroundColor
Yellow } foreach ($group in @("GRP_Finance","GRP_HR","GRP_Sales","GRP_IT")) {   New-
ADGroup -Name $group -GroupScope Global -GroupCategory Security -Path "OU=Lab Groups,
$domainDN"   Write-Host "Created group: $group" -ForegroundColor Yellow } $users = @(
@{First="John"; Last="Smith"; Username="john.smith"; Group="GRP_IT"},
@{First="Sarah"; Last="Jones"; Username="sarah.jones"; Group="GRP_Finance"},
@{First="Mike"; Last="Brown"; Username="mike.brown"; Group="GRP_Finance"},
@{First="Lisa"; Last="White"; Username="lisa.white"; Group="GRP_HR"},   @{First="Tom";
Last="Davis"; Username="tom.davis"; Group="GRP_Sales"}) foreach ($user in $users)
{   New-ADUser -GivenName $user.First -Surname $user.Last -Name "$($user.First) $
($user.Last)" `     -SamAccountName $user.Username -UserPrincipalName "$
($user.Username)@$domain" `     -Path "OU=Lab Users,$domainDN" -AccountPassword
$password -Enabled $true -PasswordNeverExpires $true   Add-ADGroupMember -Identity
$user.Group -Members $user.Username   Write-Host "Created: $($user.Username) -> $
($user.Group)" -ForegroundColor Cyan } Write-Host "`nDone." -ForegroundColor Green
```

scripts/02-configure-shares-and-permissions.ps1

Creates four folders, shares them over the network, then applies NTFS permissions per group. Share level permission is Everyone Full Control on purpose, real security comes from NTFS, not the share. (OI)(CI) means Object Inherit and Container Inherit, so files and subfolders created later automatically pick up the same permissions.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $domain = "LAB";
$basePath = "C:\Shares" New-Item -Path $basePath -ItemType Directory -Force foreach
($folder in @("Finance","HR","Sales","IT")) {   New-Item -Path "$basePath\$folder" -
ItemType Directory -Force   New-SmbShare -Name $folder -Path "$basePath\$folder" -
FullAccess "Everyone" } function Set-FolderPermissions {   param($path, $permissions)
icacls $path /inheritance:d   icacls $path /remove "BUILTIN\Users"   icacls $path
```

```
/remove "Everyone"   icacls $path /remove "NT AUTHORITY\Authenticated Users"   foreach
($p in $permissions) { icacls $path /grant "$($p.Identity):$($p.Rights)" } } Set-
FolderPermissions -path "$basePath\Finance" -permissions @(   @{Identity="$domain\
GRP_Finance"; Rights="(OI)(CI)M"},   @{Identity="$domain\GRP_HR"; Rights="(OI)(CI)R"},
@{Identity="$domain\GRP_IT"; Rights="(OI)(CI)F"},   @{Identity="BUILTIN\
Administrators"; Rights="(OI)(CI)F"}) Set-FolderPermissions -path "$basePath\HR" -
permissions @(   @{Identity="$domain\GRP_HR"; Rights="(OI)(CI)M"},
@{Identity="$domain\GRP_IT"; Rights="(OI)(CI)F"},   @{Identity="BUILTIN\
Administrators"; Rights="(OI)(CI)F"}) Set-FolderPermissions -path "$basePath\Sales" -
permissions @(   @{Identity="$domain\GRP_Sales"; Rights="(OI)(CI)M"},
@{Identity="$domain\GRP_IT"; Rights="(OI)(CI)F"},   @{Identity="BUILTIN\
Administrators"; Rights="(OI)(CI)F"}) Set-FolderPermissions -path "$basePath\IT" -
permissions @(   @{Identity="$domain\GRP_IT"; Rights="(OI)(CI)F"},
@{Identity="BUILTIN\Administrators"; Rights="(OI)(CI)F"}) Write-Host "`nDone." -
ForegroundColor Cyan
```

scripts/03-configure-rdp-gpo.ps1

Creates a Group Policy rule that turns on remote access for machines in the Lab Computers OU, and moves CLIENT01's computer account into that OU.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $gpoName = "Lab -
Allow RDP for Domain Users"; $ouPath = "OU=Lab Computers,DC=lab,DC=local" New-GPO -
Name $gpoName | Out-Null New-GPLink -Name $gpoName -Target $ouPath Set-GPRegistryValue
-Name $gpoName -Key "HKLM\System\CurrentControlSet\Control\Terminal Server" `   -
ValueName "fDenyTSConnections" -Type DWord -Value 0 $computer = Get-ADComputer -Filter
{ Name -eq "CLIENT01" } -ErrorAction SilentlyContinue if ($computer) {   $computer |
Move-ADObject -TargetPath $ouPath   Write-Host "Moved CLIENT01 to Lab Computers OU." -
ForegroundColor Cyan } else {   Write-Warning "CLIENT01 not yet in AD, may still be
joining." } Write-Host "`nGPO configuration complete." -ForegroundColor Green
```

This GPO is a backup, not something to wait on:

> • Remote access to CLIENT01 already works two other ways by the time this step runs: a earlier setup step in Step 7 adds Domain Users directly to CLIENT01's local Remote Desktop Users group. This GPO won't even apply without a policy refresh (gpupdate /force or about a 90 minute wait), so don't be alarmed if it looks like nothing happened right away.

scripts/04-domain-join.ps1

Used for both FS01 and CLIENT01. Points DNS at DC01's static address first so lab.local can be found, then retries for up to 3 minutes to give DC01 time to finish its own setup.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $adapter = Get-
NetAdapter | Where-Object {$_.Status -eq "Up"} | Select-Object -First 1 Set-
DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses
"10.0.1.4" $retries = 0; $resolved = $false do {   Start-Sleep -Seconds 15; $retries++
$resolved = [bool](Resolve-DnsName "lab.local" -ErrorAction SilentlyContinue)   Write-
Host " Attempt $retries -- resolved: $resolved" } while (-not $resolved -and $retries
-lt 12) if (-not $resolved) { throw "lab.local did not resolve after 3 minutes." }
$securePw = ConvertTo-SecureString "ADMIN_PASSWORD" -AsPlainText -Force $domainCred =
New-Object -TypeName System.Management.Automation.PSCredential -ArgumentList "LAB\
azureadmin", $securePw Add-Computer -DomainName "lab.local" -Credential $domainCred -
Restart -Force # VM restarts here. The orchestration script waits for it to come back
online.
```

scripts/05-verify-ad.ps1

Automated check of every OU, group, and user membership. Prints PASS or FAIL for each one, so you get a full status report without logging into DC01 yourself.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force Import-Module
ActiveDirectory $pass = $true Write-Host "`n=== Active Directory Verification ===" -
ForegroundColor Cyan foreach ($ou in @("Lab Users","Lab Groups","Lab Computers"))
{   $exists = [bool](Get-ADOrganizationalUnit -Filter "Name -eq '$ou'" -ErrorAction
SilentlyContinue)   if ($exists) { Write-Host " [PASS] OU: $ou" -ForegroundColor Green
}   else { Write-Host " [FAIL] OU missing: $ou" -ForegroundColor Red; $pass = $false }
} foreach ($g in @("GRP_Finance","GRP_HR","GRP_Sales","GRP_IT")) {   $exists = [bool]
(Get-ADGroup -Filter "Name -eq '$g'" -ErrorAction SilentlyContinue)   if ($exists)
{ Write-Host " [PASS] Group: $g" -ForegroundColor Green }   else { Write-Host " [FAIL]
Group missing: $g" -ForegroundColor Red; $pass = $false } } $expectedUsers =
@(   @{Username="john.smith"; Group="GRP_IT"}, @{Username="sarah.jones";
Group="GRP_Finance"},   @{Username="mike.brown"; Group="GRP_Finance"},
@{Username="lisa.white"; Group="GRP_HR"},   @{Username="tom.davis";
Group="GRP_Sales"}) foreach ($u in $expectedUsers) {   $user = Get-ADUser -Filter
"SamAccountName -eq '$($u.Username)'" -ErrorAction SilentlyContinue   if (-not $user)
{ Write-Host " [FAIL] User missing: $($u.Username)" -ForegroundColor Red; $pass =
$false; continue }   $members = Get-ADGroupMember -Identity $u.Group | Select-Object -
ExpandProperty SamAccountName   if ($members -contains $u.Username) { Write-Host "
[PASS] $($u.Username) -> $($u.Group)" -ForegroundColor Green }   else { Write-Host "
[FAIL] $($u.Username) not in $($u.Group)" -ForegroundColor Red; $pass = $false } }
Write-Host "`n=== AD Verification $(if($pass){"PASSED"}else{"FAILED"}) ===" -
ForegroundColor $(if($pass){"Green"}else{"Red"})
```

scripts/05-verify-shares.ps1

Verifies all four shares exist and every permission entry is correct, using a bitwise comparison since Windows sometimes combines multiple rights into a single value, and exact string matching would produce false failures.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $basePath = "C:\
Shares"; $pass = $true Write-Host "`n=== Share and NTFS Verification ===" -
ForegroundColor Cyan $expectedACLs = @{   "Finance" = @(     @{Identity="LAB\
GRP_Finance"; Right=[System.Security.AccessControl.FileSystemRights]::Modify},
@{Identity="LAB\GRP_HR";
Right=[System.Security.AccessControl.FileSystemRights]::Read},     @{Identity="LAB\
GRP_IT"; Right=[System.Security.AccessControl.FileSystemRights]::FullControl})   "HR"
= @(     @{Identity="LAB\GRP_HR";
Right=[System.Security.AccessControl.FileSystemRights]::Modify},     @{Identity="LAB\
GRP_IT"; Right=[System.Security.AccessControl.FileSystemRights]::FullControl})
"Sales" = @(     @{Identity="LAB\GRP_Sales";
Right=[System.Security.AccessControl.FileSystemRights]::Modify},     @{Identity="LAB\
GRP_IT"; Right=[System.Security.AccessControl.FileSystemRights]::FullControl})   "IT"
= @(     @{Identity="LAB\GRP_IT";
Right=[System.Security.AccessControl.FileSystemRights]::FullControl}) } foreach
($share in $expectedACLs.Keys) {   Write-Host "`n[ $share ]" -ForegroundColor White
$smb = Get-SmbShare -Name $share -ErrorAction SilentlyContinue   if ($smb) { Write-
Host " [PASS] SMB share exists" -ForegroundColor Green }   else { Write-Host " [FAIL]
Share missing" -ForegroundColor Red; $pass = $false; continue }   $acl = (Get-Acl
"$basePath\$share").Access   foreach ($expected in $expectedACLs[$share]) {     $ace =
$acl | Where-Object {$_.IdentityReference.Value -eq $expected.Identity -and
$_.AccessControlType -eq "Allow"}     if (-not $ace) { Write-Host " [FAIL] $
($expected.Identity) has no entry" -ForegroundColor Red; $pass = $false; continue }
$hasRight = ($ace.FileSystemRights -band $expected.Right) -eq $expected.Right     if
($hasRight) { Write-Host " [PASS] $($expected.Identity) -> $($expected.Right)" -
ForegroundColor Green }     else { Write-Host " [FAIL] $($expected.Identity) wrong
rights" -ForegroundColor Red; $pass = $false }   } } Write-Host "`n=== Verification $
(if($pass){"PASSED"}else{"FAILED"}) ===" -ForegroundColor $(if($pass)
{"Green"}else{"Red"})
```

scripts/06-add-rdp-users.ps1

Adds LAB\Domain Users to the local Remote Desktop Users group on CLIENT01, run directly through the Azure agent since WinRM is blocked by the NSG. Safe to run more than once, it checks first before adding.

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force $rdpGroup = "Remote
Desktop Users"; $domainUsers = "LAB\Domain Users" $existing = Get-LocalGroupMember -
Group $rdpGroup -ErrorAction SilentlyContinue | Where-Object {$_.Name -eq
$domainUsers} if ($existing) { Write-Host "$domainUsers already in $rdpGroup." -
ForegroundColor Yellow } else { Add-LocalGroupMember -Group $rdpGroup -Member
$domainUsers   Write-Host "Added $domainUsers to $rdpGroup." -ForegroundColor Green }
Get-LocalGroupMember -Group $rdpGroup | Select-Object Name, ObjectClass | Format-Table
-AutoSize
```

Step 7: Create the Orchestration Script

Create this one file in the project root, not inside scripts. It coordinates all seven stages by pushing each script to the right VM through the Azure agent, using az vm run-command, since the NSG blocks the ports RDP and WinRM would normally need for that.

```
param([Parameter(Mandatory=$true)][string]$KeyVaultName, [string]$ResourceGroup="RG-
FileServerLab") $startTime = Get-Date Write-Host "`n[$(Get-Date -Format "HH:mm:ss")]
Retrieving credentials from Key Vault..." -ForegroundColor Cyan $AdminPassword = az
keyvault secret show --vault-name $KeyVaultName --name "vm-admin-password" --query
"value" -o tsv if (-not $AdminPassword -or $LASTEXITCODE -ne 0) { throw "Could not
retrieve password from Key Vault. Run az login first." } Write-Host " Credentials
retrieved." -ForegroundColor Green  function Invoke-VMScript {   param([string]
$VMName, [string]$ScriptPath, [string]$Description, [hashtable]$Replacements=@{})
Write-Host "`n[$(Get-Date -Format "HH:mm:ss")] >>> $Description" -ForegroundColor Cyan
$script = Get-Content $ScriptPath -Raw   foreach ($key in $Replacements.Keys)
{ $script = $script -replace $key, [regex]::Escape($Replacements[$key]) }   $tempFile
= [System.IO.Path]::GetTempPath() + [System.IO.Path]::GetRandomFileName() + ".ps1"
$script | Out-File -FilePath $tempFile -Encoding UTF8   try {     $jsonLines = az vm
run-command invoke --resource-group $ResourceGroup --name $VMName `       --command-id
RunPowerShellScript --scripts "@$tempFile" --output json --only-show-errors     if
($LASTEXITCODE -ne 0) { throw "az vm run-command failed on $VMName" }     $result =
($jsonLines -join "`n") | ConvertFrom-Json     $stdout = ($result.value | Where-Object
{$_.code -like "*StdOut*"}).message     $stderr = ($result.value | Where-Object
{$_.code -like "*StdErr*"}).message     if ($stdout) { Write-Host $stdout }     if
($stderr -and $stderr.Trim() -ne "") { Write-Warning " VM StdErr: $stderr" }   }
finally { Remove-Item $tempFile -ErrorAction SilentlyContinue } }  function Wait-
VMOnline {   param([string]$VMName, [int]$TimeoutSeconds=360)   Write-Host " Waiting
for $VMName..." -ForegroundColor Yellow   $elapsed = 0   do {     Start-Sleep -Seconds
15; $elapsed += 15     try { $state = (az vm show -g $ResourceGroup -n $VMName -d --
query "powerState" -o tsv --only-show-errors 2>$null).Trim() } catch { $state = "" }
Write-Host " $VMName -> $state ($elapsed s)" -ForegroundColor DarkGray   } while
($state -ne "VM running" -and $elapsed -lt $TimeoutSeconds)   if ($state -ne "VM
running") { throw "Timeout: $VMName did not return within ${TimeoutSeconds}s" }
Write-Host " $VMName is online." -ForegroundColor Green }  Write-Host "`n[STEP 1]
Promoting DC01 to Domain Controller" -ForegroundColor Magenta try { Invoke-VMScript -
VMName "DC01" -ScriptPath ".\scripts\00-promote-dc.ps1" -Description "Promoting DC01"
`     -Replacements @{"SAFE_MODE_PASSWORD"=$AdminPassword} } catch { Write-Host " DC01
disconnected, expected after promotion." -ForegroundColor Yellow } Start-Sleep -
Seconds 60; Wait-VMOnline -VMName "DC01"; Start-Sleep -Seconds 90  Write-Host "`n[STEP
2] Creating OUs, Groups, and Users" -ForegroundColor Magenta Invoke-VMScript -VMName
"DC01" -ScriptPath ".\scripts\01-create-ad-users-groups.ps1" -Description "Creating AD
objects"  Write-Host "`n[STEP 3] Joining FS01 to lab.local" -ForegroundColor Magenta
Invoke-VMScript -VMName "FS01" -ScriptPath ".\scripts\04-domain-join.ps1" -Description
"Joining FS01" `   -Replacements @{"ADMIN_PASSWORD"=$AdminPassword} Start-Sleep -
Seconds 30; Wait-VMOnline -VMName "FS01"; Start-Sleep -Seconds 30  Write-Host "`n[STEP
4] Configuring shares and NTFS on FS01" -ForegroundColor Magenta Invoke-VMScript -
VMName "FS01" -ScriptPath ".\scripts\02-configure-shares-and-permissions.ps1" -
Description "Creating shares and NTFS"  Write-Host "`n[STEP 5] Joining CLIENT01 to
lab.local" -ForegroundColor Magenta Invoke-VMScript -VMName "CLIENT01" -ScriptPath ".\
scripts\04-domain-join.ps1" -Description "Joining CLIENT01" `   -Replacements
@{"ADMIN_PASSWORD"=$AdminPassword} Start-Sleep -Seconds 30; Wait-VMOnline -VMName
"CLIENT01"; Start-Sleep -Seconds 30  Write-Host "`n[STEP 5b] Granting Domain Users RDP
on CLIENT01" -ForegroundColor Magenta Invoke-VMScript -VMName "CLIENT01" -ScriptPath
".\scripts\06-add-rdp-users.ps1" -Description "Adding Domain Users to RDP group"
Write-Host "`n[STEP 6] Configuring RDP GPO on DC01" -ForegroundColor Magenta Invoke-
VMScript -VMName "DC01" -ScriptPath ".\scripts\03-configure-rdp-gpo.ps1" -Description
"Creating RDP GPO"  Write-Host "`n[STEP 7] Running automated verification" -
```

```
ForegroundColor Magenta Invoke-VMScript -VMName "DC01" -ScriptPath ".\scripts\05-verify-ad.ps1" -Description "Verifying AD" Invoke-VMScript -VMName "FS01" -ScriptPath ".\scripts\05-verify-shares.ps1" -Description "Verifying shares"  $duration = (Get-Date) - $startTime Write-Host "`n=== LAB FULLY CONFIGURED ($([math]::Round($duration.TotalMinutes,1)) min) ===" -ForegroundColor Green Write-Host "RDP into CLIENT01 as: LAB\sarah.jones Password: P@ssw0rd123!"
```

What this actually means:

- This one script does the entire build for you: promotes DC01, creates users and groups, joins the other two computers to the domain, sets up shares and permissions, grants remote access, applies the backup GPO rule, and finally checks its own work.
- az vm run-command is the channel this whole script uses to talk to each VM. It goes through the Azure agent already installed on every Azure VM, so it works even though the NSG blocks the ports RDP and WinRM would normally need.

Step 8: Run the Lab Configuration

```
.\configure-lab.ps1 -KeyVaultName "kv-fslab-XXXXXXXX" # Replace kv-fslab-XXXXXXXX with your own key_vault_name from Step 5 # Takes 15-20 minutes, fully unattended
```

Step 9: Verify the Lab

RDP into CLIENT01 using the public IP from Step 5's output. Test each scenario below. All test user passwords are P@ssw0rd123!

Log in as	Try to open	Expected result
LAB\sarah.jones	\\FS01\Finance	Read and write (member of GRP_Finance)
LAB\sarah.jones	\\FS01\HR	Access denied (not in GRP_HR)
LAB\lisa.white	\\FS01\Finance	Read only (GRP_HR has Read on Finance)
LAB\lisa.white	\\FS01\HR	Read and write (member of GRP_HR)
LAB\john.smith	\\FS01\IT	Full Control (member of GRP_IT)
LAB\tom.davis	\\FS01\Finance	Access denied (GRP_Sales has no entry)

Step 10: Pause or Tear Down

Continuing to Lab 2 (RBAC)? Stop the VMs instead of destroying them:

- Lab 2 needs RG-FileServerLab and FS01 to already exist. Stopped VMs have no compute charges, so pausing costs nothing.

```
# Pause, no compute charges while stopped az vm stop --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv) --no-wait  # Restart before Lab 2: az vm start --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv) --no-wait  # Full teardown, only when completely done with both labs: terraform destroy
```

Troubleshooting

Problem	Cause	Solution
configure-lab.ps1 fails at Key Vault	Session expired or missing role	Run az login, retry. Confirm Key Vault Secrets Officer role on the vault
Domain join fails, DNS not resolving	DC01 still finishing promotion	Wait 2 minutes and re-run, configure-lab.ps1 resumes from where it stopped
Cannot RDP to VMs	Your IP changed since deploying, or rdp_source mismatch	Run curl ifconfig.me, update terraform.tfvars rdp_source, run terraform apply

Access Denied unexpectedly	User not in the right group	Inside RDP run: whoami /groups to confirm group membership
GPO not applying	Policy cache not refreshed yet	Run gpupdate /force inside the VM, then gpresult /r. Remember RDP already works without this GPO
terraform init fails	backend.tf not updated with your storage account name	Open backend.tf, confirm the storage account name is correct and the container exists

Bonus: Words Worth Knowing

Word	What it means, in plain words
NTFS	New Technology File System, Windows' native filesystem, enforces the actual permission checks on a file or folder.
SMB	Server Message Block, the network protocol behind every Windows file share.
AD / AD DS	Active Directory / Active Directory Domain Services, the identity and directory service, the master list of every user and rule.
OU	Organizational Unit, a folder-like container in AD used to scope Group Policy to a specific set of users or computers.
GPO / GPMC	Group Policy Object / Group Policy Management Console, a rule pushed automatically to a whole group of computers at once.
ACL / ACE	Access Control List / Access Control Entry, the list of who can do what to a specific file or folder, one line at a time.
VNet / NSG	Virtual Network (a private fenced-off area) and Network Security Group (the bouncer deciding who's allowed in).
RBAC	Role-Based Access Control, giving someone a badge that matches the job they do instead of a master key to everything.
IaC	Infrastructure as Code, writing your building instructions as a file so a computer can build it for you instead of clicking through a website.
(OI)(CI)	Object Inherit / Container Inherit, icacls flags meaning any new file or folder placed inside automatically gets the same permission rules.
WinRM	Windows Remote Management, a way to run commands remotely on port 5985. Blocked on purpose in this lab, which is why az vm run-command is used instead.
RDP	Remote Desktop Protocol, the Microsoft protocol used to open a full remote screen session into a Windows machine over port 3389. This is how you log into CLIENT01 to test as each user.
GRP_ prefix	Not a technical acronym, just this lab's naming convention for its four AD security groups (GRP_Finance, GRP_HR, GRP_Sales, GRP_IT), so anyone scanning a member list can tell at a glance that it's a group and which department it maps to.
Port 3389	The network port RDP uses. The NSG in this lab only opens 3389, and only to the deployer's own IP address, which is the single doorway used to reach DC01, FS01, or CLIENT01 from outside Azure.
Port 5985	The network port WinRM uses for remote PowerShell sessions. The NSG never opens this port, which is exactly why every configuration script in this lab is pushed through az vm run-command instead of a WinRM session.

Bonus: Key Terraform Ideas Worth Understanding

Small details in the code that job interviewers tend to ask about, because they show you understand why the code is written the way it is, not just that you copied it.

Idea	Why it's there, in plain words
Remote state (backend.tf)	Instead of Terraform's notes about what it built living only on your laptop, they're saved in shared cloud storage, so any teammate's computer sees the same source of truth.
sensitive = true	Tells Terraform not to print this value on screen or in logs. A privacy screen, not a lock, it doesn't encrypt the value or stop it from ending up somewhere else if you put it in the wrong spot.
enable_rbac_authorization on Key Vault	Switches the vault to the modern permission model. Skip it and every secret read or write silently fails with a 403, even if the role assignment looks correct in the portal.

time_sleep resources	A deliberate pause after network resources are created, since Azure's control plane can have brief replication delays that cause the very next resource to fail with a transient not found error.
random_id suffix	Guarantees a globally unique name, since Key Vault names must be unique across all of Azure, not just your own subscription.
depends_on vs implicit dependency	Most ordering here comes for free from resources referencing each other's IDs. depends_on is only needed for the handful of cases, like the time_sleep pauses, where the dependency isn't expressed through a direct reference.

Bonus: Running This Lab in VS Code

Everything in this lab, the Terraform files, the PowerShell scripts, and the orchestration script, is plain text, so VS Code works well as the one place to write, edit, and run all of it instead of switching between Notepad and a separate terminal window.

What this actually means: VS Code itself does not run Terraform or Azure CLI, it is just the editor. terraform, az, and powershell still need to be installed on your machine exactly as described in the Prerequisites section. VS Code's job is to edit the files and give you an integrated terminal to run those same commands in, so nothing about the lab's steps changes, only where you type them.

Recommended extensions, install from the Extensions panel (Ctrl+Shift+X):

```
HashiCorp Terraform          syntax highlighting, formatting, and validation for .tf
files PowerShell                   syntax highlighting and debugging for .ps1 files
Azure Account                lets VS Code use the same az login session as your
terminal
```

Steps to run the lab from VS Code instead of a standalone terminal:

```
# Open the project folder created in Step 2 directly in VS Code code "$HOME\ntfs-lab-
terraform"  # Open the built in terminal inside VS Code: Ctrl+` (backtick) # It
defaults to PowerShell on Windows, the same shell used throughout this lab  #
Everything from Step 3 onward runs exactly the same way, just inside this panel: az
login terraform init terraform plan terraform apply  # .tf files get syntax
highlighting and red-underline errors from the # HashiCorp Terraform extension, which
would have caught the comma-separated # attribute bug described in the review note
near the top of this document # before ever running terraform init.
```

Everything else, editing the PowerShell scripts, running configure-lab.ps1, and RDP testing in Step 9, is identical whether it is done from VS Code or a plain terminal window. RDP itself opens outside VS Code either way, since it is a full graphical session, not something a code editor can host.

Bonus: How This Lab Would Change for a Real Company

Lab shortcut	Production grade version
Passwords set via a local environment variable	Passwords stored and rotated in Key Vault automatically, with access logged
One Key Vault Secrets Officer for one person	A small, scoped set of roles per team member, following least privilege
RDP open to one IP address	A Bastion doorway used instead, so no VM ever needs a public address at all
One domain controller	At least two, in different locations, so the domain survives one going down
You personally run configure-lab.ps1	A pipeline (like GitHub Actions) runs the whole build and verification automatically, with review steps built in